DaveBank

A Peer-to-Peer Distributed Financial Management System

Hao Yee Hew - 40412457

CSC4010 Parallel and Distributed Computing

Resit Assessment 2: Distributed Computing

27 July 2026

DaveBank is a peer-to-peer distributed financial management system written in Python using the standard library only. Every node holds a full replica of the ledger and any node can submit transactions and read balances and data item indexes, seeing results identical to every other node. Its middleware - discovery, membership, ordering, replication and recovery - is hand-written directly on UDP datagrams; no message queue, MPI, RMI or other coordination technology is used anywhere in the system. This report describes the design and logical operation of that middleware, shows how each required feature is implemented, and records how the system was tested and demonstrated.

0. Table of Contents

1. Introduction

1.a. Context and Requirements

The assignment asks for a distributed financial management system, "DaveBank", that distributes data storage and processing throughout the connected nodes of the system in a peer-to-peer architecture: accounts can be stored, transactions submitted, and current balances viewed through any connected node. Two hard constraints shape every decision that follows. The middleware must be written by hand, so no message queue, OpenMPI or RMI may handle inter-node communication or coordination; and UDP must be the primary communication protocol.

Those two constraints together are the interesting part of the problem. UDP gives no delivery guarantee, no ordering guarantee and no notion of a connection, while a bank is a system where order and consistency *are* the product. Everything in this design exists to close that gap without importing a framework that would have closed it for me.

1.b. Design Overview

The defining requirement of a banking ledger is **agreement on order**: every node must apply the same transactions in the same sequence, or balances diverge. DaveBank achieves this with **sequencer-based total-order multicast**. One node - always the live node with the lowest id, agreed by every node independently - stamps each transaction with a global sequence number and multicasts it to the network. Every node applies transactions strictly in that order, so the log, every index and every balance are identical everywhere.

The obvious objection to a sequencer is that it is a single point of failure. DaveBank answers that by making the role **automatically and deterministically re-elected**: when the sequencer dies, the next-lowest live id simply takes over, with no election traffic at all. The ordering guarantee therefore survives the loss of any node, including the one doing the ordering, which is what makes the system genuinely peer-to-peer rather than a client/server system in disguise.

The whole system is four modules of plain Python: `protocol.py` (wire format and validation), `ledger.py` (the ordered log and derived balances), `node.py` (the middleware itself) and `__main__.py` (the console). `external.py` adds an optional HTTP interface for systems outside the network.

2. System Design and Middleware

2.a. Architecture

- **Nodes** (`davebank/node.py`) are autonomous processes. Each has a stable UUID `node_id` and a user-supplied `nickname`. Every node runs the identical program; they differ only by nickname, id and port.
- **Transport** is UDP only. A main unicast socket carries all peer-to-peer traffic; a second socket bound to a shared port (50505) receives broadcast `HELLO`s for zero-configuration LAN discovery.

- **Ledger** (`davebank/ledger.py`) is an append-only log whose list position *is* the global sequence number. Balances are a pure function of that log: they are updated as each item is applied in order, and recomputed from scratch whenever the log is rebuilt, so the log remains the single source of truth.
- **Threads per node**: a main-socket receiver, a discovery-socket receiver, a maintenance loop (heartbeats, pruning, gap recovery) and the console. All shared state is guarded by locks.

2.b. Wire Protocol

Messages are small JSON objects encoded to UTF-8 bytes - human-readable, which makes a live demonstration far easier to explain, and trivial to validate. `decode()` returns `None` for anything malformed: empty or oversized packets (over 60,000 bytes), non-UTF-8 bytes, non-JSON payloads, JSON that is not an object, an unknown or absent `type`, or a missing sender id. Callers simply drop `None`, which is how malformed or hostile data is handled gracefully - a bad packet can never crash a node. Each handler is additionally wrapped so an unexpected exception on one message cannot take the node down.

Table 1 - Message types

Message	Direction	Purpose
`HELLO`	broadcast or unicast	"I exist, here is my id, nickname and port"
`WELCOME`	reply to `HELLO`	acknowledge, and pass back the full known-peer roster
`HEARTBEAT`	every node, every 2 s	liveness, log height (`seq_high`) and peer roster
`GOODBYE`	on graceful quit	leave the network immediately rather than by timeout
`SUBMIT`	origin to sequencer	please assign this transaction a sequence number
`ORDERED`	sequencer to all	transaction stamped with its global index
`STATE_REQUEST`	joiner to any peer	send me the whole ledger
`STATE_RESPONSE`	peer to joiner	a batch of the ordered log
`NACK`	any node to any peer	I am missing these sequence numbers, please resend

2.c. Membership, Discovery and Self-Healing

A node announces itself two ways: it **broadcasts** `HELLO` to the shared discovery port, and, if given `--peer HOST:PORT`, unicasts `HELLO` plus a `STATE_REQUEST` to that seed. Any recipient replies `WELCOME` carrying its full peer roster, and the newcomer introduces itself directly to every peer in that roster - so membership **gossips** to convergence with no central directory.

Liveness is maintained by a `HEARTBEAT` every 2 s; a peer unheard-from for 7 s is pruned (which covers ungraceful death), and a departing node broadcasts `GOODBYE` so peers prune it immediately (graceful disconnect).

Critically, **none of these mechanisms is one-shot**. Each heartbeat also re-broadcasts `HELLO`, carries the sender's current roster, and - while a seeded node still has no peers - retries the seed handshake. Every tick is therefore another chance for the mesh to complete itself. This was not the original design: it is a direct response to a partition bug found during the physical lab dry run (section 5.b), and it is what makes discovery robust against the single lost datagram that UDP is entitled to inflict.

2.d. Total Ordering

1. A transaction is created at its origin node with a globally-unique `item_id = "<node_id>:<local_counter>"` - unique across the network without any coordination, because the node id already is.
2. The origin sends `SUBMIT` to the current **sequencer** (if it *is* the sequencer, it orders directly).
3. The sequencer stamps the transaction with the next global sequence number `seq` and **multicasts** `ORDERED` to every peer, applying it locally too.
4. Each node applies transactions strictly in `seq` order using a **holdback queue** (`Node._deliver`): an out-of-order arrival is buffered until the gap ahead of it is filled, then the queue is drained as far as it can go.

Because every node applies the identical sequence of items, the log, every index and every balance are identical everywhere. `seq` doubles as the **network-wide index** used to retrieve data items (`get <index>`), and it means the same item on every node. Duplicates are harmless: the ledger only ever applies an item whose `seq` is exactly the next expected one, and it also tracks seen `item_id`s, so a re-delivered item is discarded rather than double-counted.

2.e. Sequencer Election and Fault Tolerance

The sequencer is, deterministically, the live peer with the **lowest `node_id`**. No election messages are exchanged at all: every node computes `min(known ids)` from its own membership view and reaches the same answer. When the sequencer dies its heartbeats stop, peers prune it after the timeout (or instantly on `GOODBYE`), and the next-lowest id simply starts ordering - resuming numbering from the highest sequence it has seen, so no index is reused or skipped. The network therefore **survives any node failing**, which is what makes it a true peer-to-peer system rather than a client/server one with extra steps.

2.f. Reliability over Unreliable UDP

UDP may drop, duplicate or reorder datagrams. DaveBank tolerates all three:

- Every heartbeat advertises the sender's log height (`seq_high`). A node that sees a height above its own, or that is holding a gap in its holdback queue, sends a **`NACK`** listing the sequence numbers it lacks. It sends that NACK to the peer whose heartbeat revealed the gap, otherwise to the sequencer, otherwise to a random peer - **any** node holding the missing items can answer, because every node holds the full log. Recovery does not depend on the sequencer being alive.
- The maintenance loop also re-issues NACKs proactively every second, so a node cannot get stuck waiting for a message that was lost.
- Duplicates and reordering are absorbed by the holdback queue and the `item_id` and expected-`seq` checks, which are idempotent.
- `--loss RATE` (or `loss RATE` at the console) deliberately drops a configurable fraction of received packets to **simulate an unreliable link**; NACK recovery then restores full consistency, which is demonstrable live.

2.g. Joining and Rebuilding

A joining node sends `STATE_REQUEST`; a peer replies with the entire log in `STATE_RESPONSE` batches. The batches are budgeted by **bytes, not item count**, because one file chunk can be tens of kilobytes while a plain transaction is tiny - this keeps every datagram under the size cap. The joiner feeds each item through the same ordered delivery path, so joining is just recovery of a very large gap. The identical path backs the `rebuild` command, which clears the local copy and reconstructs it from the network.

2.h. Binary Files as Peer-to-Peer Data Items

Rather than bolt on a file server, DaveBank carries files through the *same* replicated, totally-ordered log as money. `putfile` reads the file, computes its SHA-256 and injects (a) a `file` **manifest** item - name, size, hash, chunk count, content type, linked account - and (b) a sequence of `chunk` items, each a 30 KB slice base64-encoded (which inflates it to about 40 KB, still inside the 60 KB datagram cap).

Because these are ordinary ordered items, everything else comes for free: every node stores the file, a node joining later receives it through state-sync, and dropped chunks are recovered by exactly the same NACK mechanism as transactions. `getfile` collects the chunks belonging to a manifest, concatenates them in index order and **verifies the SHA-256 before writing**, so a missing or corrupt piece is detected rather than silently accepted. This is the brief's "full credit" route: the file *is* peer-to-peer data, with no client/server component anywhere.

2.i. Logical Networks and the External Interface

`--network NAME` puts a node on a named logical network. Every message carries its network name; a message from a foreign network is never treated as a peer, but *is* recorded, so the `networks` command lists which other DaveBank networks are audible on the LAN and how many nodes each has (entries expire after 15 s). Discovery and isolation therefore coexist: you can see networks you are not part of, and join one deliberately with `--network`.

`--api-port N` starts a small standard-library HTTP server (`external.py`) beside the node: `POST /txn` injects a transaction through the *normal* ordered path - so external input is ordered and replicated exactly like console input - while `GET /balances` and `GET /log` read state back out. Malformed JSON or missing fields return HTTP 400 rather than disturbing the node.

3. Design Rationale

3.a. Principles the Design Rests On

This section explains *why* the system is built the way it is, in terms of the standard distributed-computing principles it rests on.

- **Autonomous nodes forming a single coherent system.** Every node runs the same program and answers queries identically, giving distribution transparency for access, location, replication and failure.
- **Middleware as an overlay network.** The membership plus application-level multicast layer is an overlay: it provides broadcast-style delivery (normally unavailable across routed networks) and hides real UDP routing from the ledger and application layers.
- **Rejecting the false assumption that the network is reliable.** UDP is explicitly "fire and forget". DaveBank confronts this rather than assuming delivery: NACK-based retransmission, repeated discovery, and a deliberate packet-loss simulator used as a test instrument.
- **Replication consistency.** Instead of letting replicas diverge and asking "which copy is right?", the design gives every node the same single source of truth by agreeing one total order and applying it idempotently.
- **Ordering, happened-before and logical clocks.** A bank needs a *total* order, which is stronger than the partial order a bare Lamport clock gives. We obtain it by routing every transaction through one **sequencer** that stamps a monotonic global sequence number - conceptually a Lamport clock centralised at a single node, so ties are impossible.
- **Coordinator, election, and the centralisation trade-off.** The sequencer is an algorithmically chosen coordinator, or super-peer. Centralising the decision is good (simple, one source of truth) and bad (single point of failure); we keep the good and remove the bad by **electing** a replacement when the coordinator dies. "Lowest live id wins" is a deterministic election in the same family as the classic bully and ring algorithms, but needs **zero election messages** because every node computes the same winner from its own membership view - cheaper, and with no election-storm failure mode.

- **Discovery by flooding and gossip.** Nodes are learned through broadcast `HELLO` and gossiped rosters - the unstructured peer-to-peer flooding pattern.
- **Chunked content addressed by hash.** The file feature mirrors BitTorrent's manifest and piece model: a manifest describes the pieces, the pieces travel independently, and the hash proves the reassembly.

3.b. Trade-offs Accepted

Every design decision above costs something. These are the costs, stated plainly.

- **Full replication instead of sharding.** Distributing data around a ring (Chord-style consistent hashing) would conflict with this design's guarantee that *every* node answers *every* query from a complete local replica, and it would weaken resilience at this scale: with a handful of nodes, losing one shard owner loses data, whereas full replication survives the loss of all but one node. It is the right trade at ten nodes and the wrong one at ten thousand, where a production system would shard *and* replicate each shard.
- **A brief changeover window during re-election.** If two nodes momentarily disagree about membership, both could believe they are the sequencer. Numbering resumes from the highest sequence seen, which limits the damage, but a quorum protocol (Raft, Paxos) would close the gap properly.
- **The ledger is in-memory.** State survives any node failing, because the survivors hold it and a restarted node rebuilds from them - but not the whole network stopping at once. Persisting the log to disk would remove that limit.
- **No authentication or encryption.** Anyone on the LAN could inject a transaction. For a real bank each item would be signed with a per-node key; that is out of scope here, but the ordered-log design would carry signatures without structural change.

4. Features Implemented

Every requirement from the brief, how it is implemented, and how to see it running. All commands are typed at a node's console unless stated otherwise.

4.a. Must Have (30%)

Table 2 - Must have requirements

Requirement	Implementation	How to see it
Connect at least two nodes	Broadcast `HELLO` + `--peer` seed + gossiped rosters (`node.py`)	Start 2 nodes, type `peers`
Send numeric account transactions	`deposit` / `withdraw`, validated in `Transaction.from_dict`	`deposit alice 100`
Index for data items	`Transaction.seq`, assigned by the sequencer; list position in the log	`log`
Retrieve item by index on any node	`Ledger.get(seq)`	`get 1` on a different node
Retrieve an account balance on any node	Balances derived from the ordered log	`balance alice`, `balances`
Friendly nickname shown by all nodes	`--nick`, carried in every message	`peers` on any node
Uniquely identify all nodes	UUID4 `node_id` per process	`whoami`, `peers`
Same indexes on every node	Guaranteed by total order + holdback queue	`log` side by side on all nodes
Uniquely identify all data items	`item_id = "<node_id>:<counter>"`	`get <index>` shows the item id
UDP as the primary protocol	Entire transport is `SOCK_DGRAM`; no TCP between nodes	Whole system
Gracefully handle disconnecting nodes	`GOODBYE` broadcast on quit, plus 7 s heartbeat timeout for crashes	`quit` one node, `peers` elsewhere
Demonstrate on multiple machines	Binds `0.0.0.0`; join with the real IP in `--peer`	Demonstrated on CSB lab machines

4.b. Should Have (30%)

Table 3 - Should have requirements

Requirement	Implementation	How to see it
Multiple synchronised nodes	Full replication through ordered multicast	`balances` identical on every node
New nodes can join	`STATE_REQUEST` / `STATE_RESPONSE` full-log transfer	Start a node late, then `log`
Complete accurate ordering, consistent everywhere	Sequencer-assigned `seq` + holdback queue	Submit from 3 nodes, compare `log`
Balances for multiple accounts	Arbitrary account names, one map derived from the log	`balances`
A joining node receives all data items	Byte-budgeted state-sync batches	`log` on the new node
Robot to generate example data	Background generator, random deposit/withdraw/transfer	`robot start`, `robot stop`
Handle malformed data gracefully	`protocol.decode` + `Transaction.from_dict` validation; per-message exception guard	Bad packets and bad HTTP posts are dropped, node survives
True P2P coping with any node failing	Deterministic re-election of the lowest live id	`quit` the `*seq` node; survivors keep working
Simulate unreliable links / disconnects	`--loss` / `loss RATE` drops a fraction of packets	`loss 0.5`, watch the lag, then recovery
Demonstrate on multiple physical lab machines	Single-file `project.exe` run from the whitelisted lab path	Demonstrated on CSB lab machines
More complex data items	`transfer` (two-account atomic move), plus `file` and `chunk` items	`transfer alice bob 30`

The robot draws from a fixed pool of fictional accounts - `alice`, `bob`, `carol`, `dave`, and, in tribute to DaveBank's regulator and auditor as named in the brief, `enron` and `lehman`.

4.c. Could Have (20%)

Table 4 - Could have requirements

Requirement	Implementation	How to see it
Discovery of nodes	Broadcast `HELLO` on a shared port, gossiped rosters, `--peer` seed, all repeated every heartbeat	`peers` converges with no configuration
Discovery of a network / networks to join	Named logical networks; foreign networks overheard and listed	`--network testnet`, then `networks`
Discover and obtain missing data, and simulate it	`seq_high` in heartbeats, `NACK` recovery, `--loss` simulator	`loss 0.6`, submit elsewhere, watch it catch up
Transfer binary files linked in data items	**Full-credit route**: file manifest + base64 chunks as ordered P2P data items, SHA-256 verified (section 2.h)	`putfile statement.png`, `files`, `getfile 0 copy.png`
Clear local copy and rebuild from the network	`Ledger.rebuild_from` + state-sync	`rebuild`, then `log`
External interface	Standard-library HTTP API: `POST /txn`, `GET /balances`, `GET /log`	`--api-port 8080`, then post from PowerShell or curl
Distribute data around the network for resilience	**Deliberately not implemented** - full replication is the stronger trade at this scale (section 3.b)	-

5. Testing and Verification

5.a. Automated Test Harness

Correctness here is not "it looked right on screen": an integration harness (`tests/integration_smoke.py`) asserts it, and it passes repeatably. The harness spins up real UDP nodes inside one process and checks:

- **Membership convergence** - three nodes started independently discover each other and agree on the same sequencer.
- **Total ordering** - five transactions submitted from different nodes land in the identical order with identical balances on every node.
- **Join-sync** - a node started after the fact receives the whole ledger.
- **Sequencer failover** - killing the sequencer re-elects the next-lowest id, and the survivors keep applying new transactions consistently.

- **NACK recovery** - under simulated packet loss a lagging node detects the gap and recovers to a fully consistent ledger once the loss subsides.

The advanced features are exercised the same way: an upload is checked by retrieving it on a *different* node and confirming the SHA-256 matches byte-for-byte, an external HTTP post is confirmed to reach other nodes over UDP while a malformed post is rejected without disturbing the node, and a second logical network is confirmed to be discoverable while staying out of `peers`.

5.b. Bugs Found and Fixed

Each of these was found by testing rather than by reading code, and each is a genuinely distributed failure mode worth recording.

(a) A departed peer could kill a node's receiver (Windows). Sending a UDP datagram to a peer that has gone produces an ICMP port-unreachable, which surfaces as `ConnectionResetError` on the *sender's* next `recvfrom`. The receive loops originally treated any `OSError` as fatal and exited, so a node's receiver thread died silently when a peer left - precisely when failover most needed it. Fixed: the loops only exit if the node is actually shutting down.

(b) Graceful `GOODBYE` was never sent. `stop()` set the shutdown flag before broadcasting `GOODBYE`, and the send path short-circuits once that flag is set, so the announcement was suppressed and peers only noticed a departure via the 7 s timeout. Fixed by broadcasting first, then setting the flag: departures are now pruned instantly.

(c) The mesh could partition permanently. Found on real lab machines: two nodes each joined the seed successfully, yet never learned about each other. The cause was two independent *one-shot* mechanisms. The seed handshake was sent exactly once at start-up, so a single lost datagram - entirely plausible for the very first packet between two machines - stranded a node with no peers forever; and peers were introduced to each other only via the roster attached to a `WELCOME`, so when two nodes joined the same seed at nearly the same instant, neither's roster yet contained the other and nothing re-triggered the introduction. The fix is the design principle now described in section 2.c: **no membership mechanism is one-shot**. The seed handshake retries every tick until a peer registers, and every heartbeat carries a full roster which the receiver uses to introduce itself to anyone new. A targeted test that forced the exact stuck state confirmed the mesh then self-heals in about 0.2 s, and the fix was verified again on the lab machines themselves.

6. Wow Factor

- **Zero-message election.** The coordinator is agreed with no election traffic and no consensus round at all: it falls out of the membership view every node already maintains. Failover costs nothing and cannot storm.
- **Self-healing membership.** Every heartbeat re-runs discovery, so the mesh repairs partitions continuously instead of depending on a lucky first packet (section 5.b) - the property that turned a fragile demo into a reliable one.
- **Files as first-class P2P data items.** Binary content rides the same totally-ordered log as money, so replication, late-join delivery, loss recovery and integrity checking are all inherited rather than

reimplemented.

- **Failure simulation as a built-in instrument.** `--loss` makes the unreliable network reproducible on demand, and is used by the automated tests, not just the demonstration.
- **Verification, not vibes.** The tricky properties - total order, failover, loss recovery - are asserted by an automated harness against real UDP nodes, not eyeballed on screen.
- **Deployment engineering for the lab reality.** The system ships as a single PyInstaller executable so it can run from the firewall-whitelisted path on the locked-down CSB machines, where no admin account exists to approve a socket prompt - the practical hurdle the brief warns about.

7. Running the System

From the folder containing the `davebank` package (Python 3.8+, no third-party packages required):

```
python -m davebank --nick alice --port 50000
python -m davebank --nick bob   --peer 192.168.1.10:50000
python -m davebank --nick carol --peer 192.168.1.10:50000 --api-port 8080
```

Useful options: `--network NAME` (logical network), `--loss 0..1` (simulate an unreliable link), `--robot` (start the data generator immediately), `--api-port N` (external HTTP interface; must be set at start-up).

Console commands: `deposit`, `withdraw`, `transfer`, `balance`, `balances`, `get <index>`, `log`, `peers`, `networks`, `whoami`, `putfile`, `files`, `getfile`, `robot start|stop`, `loss <rate>`, `sync`, `rebuild`, `quit`.

For the lab machines the system is also built as a single self-contained executable (`pyinstaller --onefile --name project davebank_main.py`), run as `C:\temp\project\project.exe --nick alice --port 50000`.